

19. RowSet interface

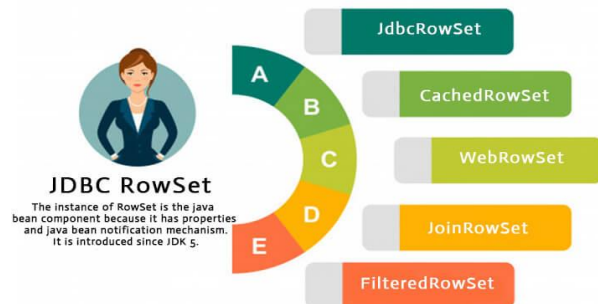
An instance of RowSet is the Java bean component because it has properties and Java bean notification mechanism. It is the wrapper of ResultSet. A JDBC RowSet facilitates a mechanism to keep the data in tabular form. It happens to make the data more flexible as well as easier as compared to a ResultSet. The connection between the data source and the RowSet object is maintained throughout its life cycle. The RowSet supports development models that are component-based such as JavaBeans, with the standard set of properties and the mechanism of event notification.

It was in the JDBC 2.0, the support for the RowSet was introduced using the optional packages. But the implementations were standardized for RowSet in the JDBC RowSet Implementations Specification (JSR-114) by the Sun Microsystems that is being present in the JDK (Java Development Kit) 5.0.

Types of RowSet Interfaces

There are 5 major types sub-interfaces of RowSet interface.

- JdbcRowSet
- CachedRowSet
- WebRowSet
- JoinRowSet
- FilteredRowSet



Advantage of RowSet

The advantages of using RowSet are given below:

- It is easy and flexible to use.
- It is Scrollable and Updatable by default.

19.1 JdbcRowSet

- **Definition:** A JdbcRowSet is a connected RowSet that works directly with a JDBC connection. It maintains a live connection to the database, similar to a ResultSet.
- **Features:**
 - Connected to the database while iterating through data.
 - Extends ResultSet and is ideal for applications that require a live connection for frequent updates.
 - Implements the RowSet interface.
 - Can be used as a JavaBean component, supporting properties like event listeners.
- **Use Case:**
 - For small data manipulations where a live connection is necessary.

Example for Performing Basic CRUD operations using RowSet

```

package com.lokesh;

import javax.sql.rowset.*;
import java.sql.*;

// This class demonstrates use of basic functionality of rowsets
public class RowSetCRUDExample {

    static final String DB_URL = "jdbc:mysql://localhost/organization";
    static final String USER = "root";
    static final String PASS = "root";

    public static void main(String args[]) throws SQLException {
        JdbcRowSet rowSet = RowSetProvider.newFactory().createJdbcRowSet();
        rowSet.setUrl(DB_URL);
        rowSet.setUsername(USER);
        rowSet.setPassword(PASS);
        rowSet.setCommand("select id, age, first, last from employees");
        rowSet.execute();

        while (rowSet.next()) {
            System.out.print("Id: " + rowSet.getInt("id"));
            System.out.print(" Age: " + rowSet.getInt("age"));
            System.out.print(" First: " + rowSet.getString("first"));
            System.out.println(" Last: " + rowSet.getString("last"));
        }

        System.out.println("-----");
        // Position the cursor to the 3rd row
        rowSet.absolute(3);
        rowSet.updateInt("age", 20);
        rowSet.updateRow();

        // position the cursor to the last row
        rowSet.last();
        rowSet.deleteRow();

        // After updating the 3rd row and deleting the last row, doing a select to
        // view

        // updated records.
        rowSet.setCommand("select id, age, first, last from employees");
        rowSet.execute();

        while (rowSet.next()) {
            System.out.print("Id: " + rowSet.getInt("id"));
            System.out.print(" Age: " + rowSet.getInt("age"));
            System.out.print(" First: " + rowSet.getString("first"));
            System.out.println(" Last: " + rowSet.getString("last"));
        }
    }
}

```

Output:

```

Id: 1 Age: 20 First: Rita Last: Tez
Id: 2 Age: 20 First: Sita Last: Singh
Id: 3 Age: 35 First: Zaid Last: Khan
Id: 4 Age: 33 First: Sumit Last: Mittal
Id: 5 Age: 40 First: John Last: Paul
Id: 7 Age: 20 First: Sita Last: Singh
Id: 8 Age: 20 First: Rita Last: Tez
-----
Id: 1 Age: 20 First: Rita Last: Tez

```

```

Id: 2 Age: 20 First: Sita Last: Singh
Id: 3 Age: 20 First: Zaid Last: Khan
Id: 4 Age: 33 First: Sumit Last: Mittal
Id: 5 Age: 40 First: John Last: Paul
Id: 7 Age: 20 First: Sita Last: Singh

```

Example of JDBC RowSet with Event Handling

To perform event handling with `JdbcRowSet`, you need to add the instance of `RowSetListener` in the `addRowSetListener` method of `JdbcRowSet`.

The `RowSetListener` interface provides 3 method that must be implemented. They are as follows:

1. `public void cursorMoved(RowSetEvent event);`
2. `public void rowChanged(RowSetEvent event);`
3. `public void rowSetChanged(RowSetEvent event);`

Let's write the code to retrieve the data and perform some additional tasks while the cursor is moved, the cursor is changed, or the rowset is changed. The event handling operation can't be performed using `ResultSet`, so it is preferred now.

```

package com.lokesh;

import javax.sql.rowset.*;
import javax.sql.*;
import java.sql.*;

// This class demonstrates use of RowSetEvent and event-handling in RowSet
public class RowSetEventHandlingExample {

    static final String DB_URL = "jdbc:mysql://localhost/organization";
    static final String USER = "root";
    static final String PASS = "root";

    public static void main(String args[]) throws SQLException {

        JdbcRowSet rowSet = RowSetProvider.newFactory().createJdbcRowSet();
        rowSet.setUrl(DB_URL);
        rowSet.setUsername(USER);
        rowSet.setPassword(PASS);
        rowSet.setCommand("select id, age, first, last from employees");
        rowSet.execute();
        rowSet.addRowSetListener(new CustomListener());

        while (rowSet.next()) {
            // Generating cursor Moved event
            System.out.print("Id: " + rowSet.getInt(1));
            System.out.print(" Age: " + rowSet.getInt("age"));
            System.out.print(" First: " + rowSet.getString("first"));
        }

        System.out.println("-----");
        rowSet.absolute(3);
        rowSet.updateInt("age", 30);
        // Generating row changed event
        rowSet.updateRow();
    }
}

```

```

    }
}

class CustomListener implements RowSetListener {
    public void cursorMoved(RowSetEvent evt) {
        System.out.println(" Cursor Moved...");
    }

    public void rowChanged(RowSetEvent evt) {
        System.out.println(" Row Changed...");
    }

    public void rowSetChanged(RowSetEvent evt) {
        System.out.println(" RowSet changed..");
    }
}

```

Output:

```

Cursor Moved...
Id: 1 Age: 20 First: Rita Cursor Moved...
Id: 2 Age: 20 First: Sita Cursor Moved...
Id: 3 Age: 30 First: Zaid Cursor Moved...
Id: 4 Age: 33 First: Sumit Cursor Moved...
Id: 5 Age: 40 First: John Cursor Moved...
Id: 7 Age: 20 First: Sita Cursor Moved...
-----
Cursor Moved...
Row Changed...

```

19.2 CachedRowSet

- **Definition:** A CachedRowSet is a disconnected RowSet that caches its data in memory. After fetching data from the database, it closes the connection.
- **Features:**
 - Disconnected from the database after data retrieval, reducing database resource usage.
 - Serializable and can be sent over a network.
 - Data can be updated offline and synchronized back to the database.
 - Supports scrolling and updating rows.
- **Use Case:**
 - For applications requiring offline data manipulation.

Example:

```

CachedRowSet cachedRowSet = RowSetProvider.newFactory().createCachedRowSet();
cachedRowSet.setUrl("jdbc:mysql://localhost:3306/mydb");
cachedRowSet.setUsername("user");
cachedRowSet.setPassword("password");
cachedRowSet.setCommand("SELECT * FROM employees");
cachedRowSet.execute();

while (cachedRowSet.next()) {
    System.out.println(cachedRowSet.getString("name"));
}

```

19.3 WebRowSet

- **Definition:** A WebRowSet is a special type of CachedRowSet that supports XML data representation. It provides XML read/write capabilities.
- **Features:**
 - Can read and write data in XML format.
 - Ideal for exchanging data between different applications or systems.
 - Extends CachedRowSet and includes methods for XML input/output.
- **Use Case:**
 - When data needs to be exchanged over the web or stored in XML format.

Example:

```
WebRowSet webRowSet = RowSetProvider.newFactory().createWebRowSet();
webRowSet.setUrl("jdbc:mysql://localhost:3306/mydb");
webRowSet.setUsername("user");
webRowSet.setPassword("password");
webRowSet.setCommand("SELECT * FROM employees");
webRowSet.execute();

// Write to XML
webRowSet.writeXml(System.out);

// Read from XML
webRowSet.readXml(new FileReader("employees.xml"));
```

19.4 JoinRowSet

- **Definition:** A JoinRowSet allows joining multiple RowSet objects. It behaves like a SQL JOIN operation.
- **Features:**
 - Can join data from different RowSets based on a common column.
 - Combines data in a way similar to SQL joins but in memory.
- **Use Case:**

To perform in-memory joins when data is fetched from different sources.

```
CachedRowSet rowSet1 = RowSetProvider.newFactory().createCachedRowSet();
rowSet1.setCommand("SELECT id, name FROM employees");
rowSet1.execute();

CachedRowSet rowSet2 = RowSetProvider.newFactory().createCachedRowSet();
rowSet2.setCommand("SELECT id, department FROM departments");
rowSet2.execute();

JoinRowSet joinRowSet = RowSetProvider.newFactory().createJoinRowSet();
joinRowSet.addRowSet(rowSet1, "id");
joinRowSet.addRowSet(rowSet2, "id");

while (joinRowSet.next()) {
    System.out.println(joinRowSet.getString("name") + " - " +
        joinRowSet.getString("department"));
}
```

19.5 FilteredRowSet

- **Definition:** A FilteredRowSet is a disconnected RowSet that allows filtering data based on a custom condition.
- **Features:**
 - Extends CachedRowSet and enables filtering rows using a predicate or filter criteria.
 - Implements the Predicate interface for filtering.
- **Use Case:**
 - To retrieve or manipulate a subset of data based on conditions.

Example:

```
FilteredRowSet filteredRowSet = RowSetProvider.newFactory().createFilteredRowSet();
filteredRowSet.setCommand("SELECT * FROM employees");
filteredRowSet.execute();

// Custom filter to show employees with salary > 50000
filteredRowSet.setFilter(row -> {
    try {
        return row.getInt("salary") > 50000;
    } catch (SQLException e) {
        return false;
    }
});

while (filteredRowSet.next()) {
    System.out.println(filteredRowSet.getString("name"));
}
```

19.6 Summary of Key Differences

Type	Connection	Data Storage	Specialty	Use Case
JdbcRowSet	Connected	Live	Works with a live database connection.	Frequent updates in a connected mode.
CachedRowSet	Disconnected	Cached	Stores data in memory for offline use.	Offline data manipulation.
WebRowSet	Disconnected	Cached	Supports XML input/output.	Web data exchange in XML format.
JoinRowSet	Disconnected	Cached	Joins multiple RowSets.	In-memory SQL-like joins.
FilteredRowSet	Disconnected	Cached	Filters rows based on criteria.	Subset of data based on conditions.

Each RowSet type offers unique advantages tailored to specific use cases in Java database programming.